이음 E:UM

배치 실행·서버 관제·업무 관제를 하나로 잇는 시스템 운영 도구

사건은 하나인데, 도구는 여럿입니다.

- 장애 한 건을 분석하려면 운영자의 경험과 숙련도에 의존하여 여러 도구를 뒤져봐야 합니다.
- 서버도, 데이터 저장소도 따로입니다. 데이터는 서로 엮이지 않고 분석을 위해 다루기 어렵습니다.
- 어느 코드가 장애와 지연을 유발하는지 어떤 도구에도 남지 않습니다.

> **이음은 여러 도구를 하나로 합치고, 실행마다 `실행 번호`를 붙여 모든 기록을 잇습니다.**

사용자: 운영 담당자·개발자 대상: 폐쇄망 리눅스 서버

결과물: 하루 만에 만드는 검증판

Q1 그래서 대체 뭘 만드는 건가요?

스케줄러, 서버 관제, 함수 호출 레벨의 업무 관제 도구를 실행 파일 하나로 통합합니다.

2~3쪽

Q2 기존의 뭘 보완하는 건가요?

비어 있는 도구 사이. 도구끼리 이어지지 않고, 데이터는 제품 안에 갇혀 있고, 비용은 도구마다 나갑니다.

4~6쪽

Q3 뭐가 좋아지는 건가요?

시스템의 복잡도와 관리 비용이 줄고, 배치를 유연하게 실행하고, 장애 원인을 한눈에 파악하고, 수집된 데이터를 마이닝할 수 있는 가능성을 열어 줍니다.

7~13쪽

Q4 어떻게 가능한 건가요?

여러 도구를 하나의 키로 잇습니다. 단순화된 아키텍처의 확고한 기반을 2명이 AI 세션 4개로 구현·입증합니다. 확장 가능한 인터페이스와 모듈 설계로, 시간이 허락하는 한 더 정교한 기능을 얼마든지 추가할 수 있도록 계획합니다.

14~15쪽

Q1 그래서 대체 뭘 만드는 건가요?

도구 여러 개가 하던 일을 실행 파일 하나로.

스케줄러·서버 관제·업무 관제의 추적 수준을 강화하여 하나로 통합합니다.

배치 스케줄러
정해진 시각에 작업 실행
지금: 상용 Control-M

서버 관제
서버·프로세스의 CPU·메모리
지금: 상용 onTune

업무 관제 업무 프로세스 추적
주문 접수·처리·응답시간
+ 함수 호출 단위까지
지금: 사내 APM 함수 단위 없음

→

이음 + 표시가 강화한 추적

- 실행: 플로우·재시도 **+ 띄운 서비스까지** — 번호
- 자원: 서버 **+ 실행별** CPU·메모리 — 번호
- 업무: 접수·처리·응답 **+ 함수 호출** — 번호

통합 개방 DB : 실행 이력 · 서버 자원 · 실행별 자원 · 업무 지표 · 함수 기록

웹 화면	SQL 조회	내보내기
대시보드·플로우·타임머신	읽기 전용	CSV·함수 기록

설치 구성

에이전트 1
서버당
실행·수집·함수 기록

+

매니저 1
DB 저장·스케줄링·화면
에이전트를 모아 관리

=

E:UM
실행 파일 하나

같은 실행 파일을 역할만 바꿔 띄웁니다.

운영 요구	이음의 방식
폐쇄망	외부 서비스·CDN 없이 동작
방화벽	에이전트 → 매니저 한 방향. 포트 1개
실행 통제	검토된 작업 정의(YAML)의 명령만 실행. 토큰 인증
서버 부담	함수 추적은 지정한 작업만. 추적 부하도 화면에 표시

한 줄 정의

작업을 띄우는 도구가 지켜보는 일까지 맡습니다.

Q1 그래서 대체 뭘 만드는 건가요? 운영 흐름으로 보기

띄우고, 지켜보고, 돌아보는 일을 번호 하나로 잇습니다.

지금은 단계마다 도구가 바뀌고, 도구끼리 기록이 이어지지 않습니다.



① 띄운다

개장 준비 플로우가 서비스 기동을 책임지고 번호를 발급합니다.

지금은 Control-M

② 지켜본다

시스템 자원 사용 상황, 서비스의 실행 콜스택 정보를 수집하고 추적합니다. 띄운 스크립트가 끝나도 서비스는 같은 번호로 계속 기록됩니다.

지금은 onTune + 사내 APM, 함수 기록은 없음

③ 돌아본다

지난 구간을 다시 펼쳐 함수까지 확인합니다. SQL·CSV로 용량과 배포 전후를 분석합니다.

지금은 제품 화면 안에서만

띄우는 도구가 지켜보는 일까지 맡아야 **실행 번호**가 끊기지 않습니다. 여러 도구를 하나로 합치는 이유입니다.

Q2 기존의 뭘 보완하는 건가요?

비어 있는 도구 사이를 하나로 잇습니다.

도구마다 제 몫은 합니다. 하지만 도구를 잇는 공통 키가 없고, 어느 코드가 CPU를 썼는지는 어느 도구도 모릅니다.

배치 스케줄러

상용 · Control-M

무엇을 언제 실행했고, 성공했나

단위: 작업

자체 화면 · 자체 저장소

서버 관제

상용 · onTune

어느 서버·프로세스가 CPU·메모리를 썼나

단위: 서버·프로세스

자체 화면 · 자체 저장소

업무 관제

사내 개발 APM

주문이 몇 건 들어와 처리됐고, 응답은 얼마나 걸렸나

단위: 업무 프로세스

자체 화면 · 자체 저장소

공통 키 없음 → 사람이 시각·서버 이름으로 맞춤

함수(코드) 단위 기록 없음 → 필요하면 개발자가 서버에서 직접 측정

데이터는 **제품마다 따로**. 형식·보존 기간·추출 범위는 제품이 정함

비어 있는 곳과 이음이 채우는 것

도구	비어 있는 곳	이음이 채우는 것
배치 스케줄러	띄운 서비스의 이후 상태를 모름. 업무가 바빠도 시각이 되면 실행	**태어난 서비스는 죽을 때까지 정밀 감시당합니다.**
서버 관제	프로세스를 띄운 실행, CPU를 쓴 함수를 모름	**서버 리소스를 축내는 서비스는 그 자리에서 즉시 코드 레벨까지 파헤쳐집니다.**
업무 관제	지연 원인이 업무량·자원·코드 중 무엇인지 모름	**업무가 밀리면 업무량·자원·함수가 한 시간축에 펼쳐집니다.**
도구 공통	데이터는 제품 안에. 계약·서버·화면이 도구마다 따로	**통합·개방된 DB와 이를 다루는 실행 파일 하나. 극도의 단순함을 추구합니다.**

보완할 곳

도구 사이의 연결, 코드 단위 기록, 갇힌 데이터.

Q2 기존의 뭘 보완하는 건가요? 빈틈이 만드는 불편

빈틈이 만드는 불편은 세 가지입니다.

불편 1

특이상황 확인이 번거롭다

장중 체결 지연을 지금 따라가는 순서. 어디부터 볼지는 운영자 경험에 달려 있습니다.

1	업무 관제	응답시간 상승, 처리 대기 증가 확인
2	운영 담당	해당 서비스의 서버·프로세스 확인
3	서버 관제	해당 시각 CPU 확인. "CPU를 다 씀"에서 멈춤
4	스케줄러	서비스를 띄운 실행, 같은 시각 다른 작업 확인
5	개발자	폭주는 이미 끝남. **부하를 다시 만들어 재현** 후 측정

여러 도구, 5단계. 원인 코드는 재현부터

불편 2

쌓인 데이터를 쓰기 어렵다

운영 질문 대부분은 두 도구 이상의 데이터를 엮어야 답이 나옵니다.

- 초당 몇 건부터 체결이 밀리나? 증설할 때인가?
  업무 지표 + 자원
- 밀린 구간에 CPU를 쓴 함수는 평소와 뭐가 다른가?
  업무 지표 + 함수 기록(지금은 없음)
- 정산 배치가 끝나는 시각이 매달 늦어지나?
  실행 이력 + 자원
- 배포 뒤 같은 작업의 함수별 CPU 비중이 바뀌었나?
  실행 이력 + 함수 기록

통계·머신러닝도 엮인 원자료가 있어야 시작

불편 3

비용이 많이 든다

매일 쓰는 기능은 일부입니다. 비용은 제품 단위로 나갑니다.

스케줄러 라이선스·유지보수	상용 · 매년
서버 관제 라이선스·유지보수	상용 · 매년
업무 관제(사내 APM) 유지보수	사내 인력
제품별 관리 서버·저장소	도구마다
계약 갱신·교육·문의 창구	제품마다

제품에 없는 기능은 벤더 개발이나 별도 연동을 기다려야 합니다.

쓰는 건 일부, 비용은 도구마다

Q2 기존의 뭘 보완하는 건가요? 보완 방법

연동으로는 못 메웁니다. 하나로 새로 만듭니다.

보완 방법 네 가지를 비교했습니다. 기준은 두 가지. 실행 번호로 기록을 이을 수 있는가, 데이터를 우리가 쥐는가.

방법	실행 번호로 잇기	함수 단위 기록	데이터를 SQL로 직접	비용	폐쇄망·기능 추가
A. 지금 도구 + 연동	△ 번호를 주고받지 못해 시각으로 맞춤	×	△ 제품별 형식	높음 기존 + 연동비	△ 벤더 일정
B. 오픈소스 조합 스케줄러+관제+프로파일러	△ 잇는 코드를 결국 직접 작성	△ 별도 도구	○ 저장소 3개	낮음	△ 제품 3개 운영
C. 통합 관측 제품 상용 APM·관측 플랫폼	△ 배치 스케줄러 없음	△ 제품마다 다름	△ 벤더 저장소	매우 높음	△ 폐쇄망 제약
D. 이음	○ 띄울 때 번호 부여	○ 같은 번호·같은 DB	○ 공개 스키마	낮음 서버 + 인건비	○ 코드 사내 보유

B도 결국 여러 제품을 잇는 데이터 모델과 연동 코드를 직접 만들어야 합니다. 그 부분이 이 과제의 핵심이라 처음부터 하나로 만듭니다.

번호는 띄울 때만 붙일 수 있습니다

택배 송장번호와 같습니다. 보낼 때 붙여야 이후 기록이 조회됩니다. 배치가 서비스를 띄울 때 번호를 넘겨야 자원·업무 지표·함수 기록이 같은 번호를 답니다.

자체 구현 부담도 셉니다

유지보수는 우리 몫입니다. 비용 비교에 인건비를 넣고, 검증된 범위부터 병행 운영합니다(13쪽).

결론

띄우는 쪽(스케줄러)과 재는 쪽(관제)이 한 제품이어야 번호가 끊기지 않습니다.

Q3 뭐가 좋아지는 건가요?

좋아지는 건 네 가지입니다.

장애 대응만 달라지지 않습니다. 평소 운영, 배치 관리, 데이터 활용이 함께 달라집니다.

① 복잡도와 관리 비용이 줄어듭니다

지금 도구마다 계약·관리 서버·화면이 따로

이음 **실행 파일 1개, 화면 1개, 방화벽 포트 1개**

화면 ① 대시보드 · 8쪽

② 배치를 유연하게 실행합니다

지금 작업 목록·로그 위주. 업무가 바빠도 시각이 되면 실행

이음 **병렬·합류·서버 넘기·재시도를 그래프로. "체결 대기 0건 뒤 정산" 같은 조건 실행**

확장

화면 ② 플로우 그래프 · 9쪽

③ 장애 원인을 한눈에 파악합니다

지금 여러 도구, 5단계. 원인 코드는 재현 후에

이음 **화면 1개, 2단계. 재현 없이 함수까지**

화면 ③ 타임머신 · ④ 플레임 그래프 · 10~11쪽

④ 쌓인 데이터를 마이닝합니다

지금 제품 화면·보고서 안에서만 확인

이음 **운영 데이터 5종을 SQL로 조인. CSV로 통계·머신러닝에**

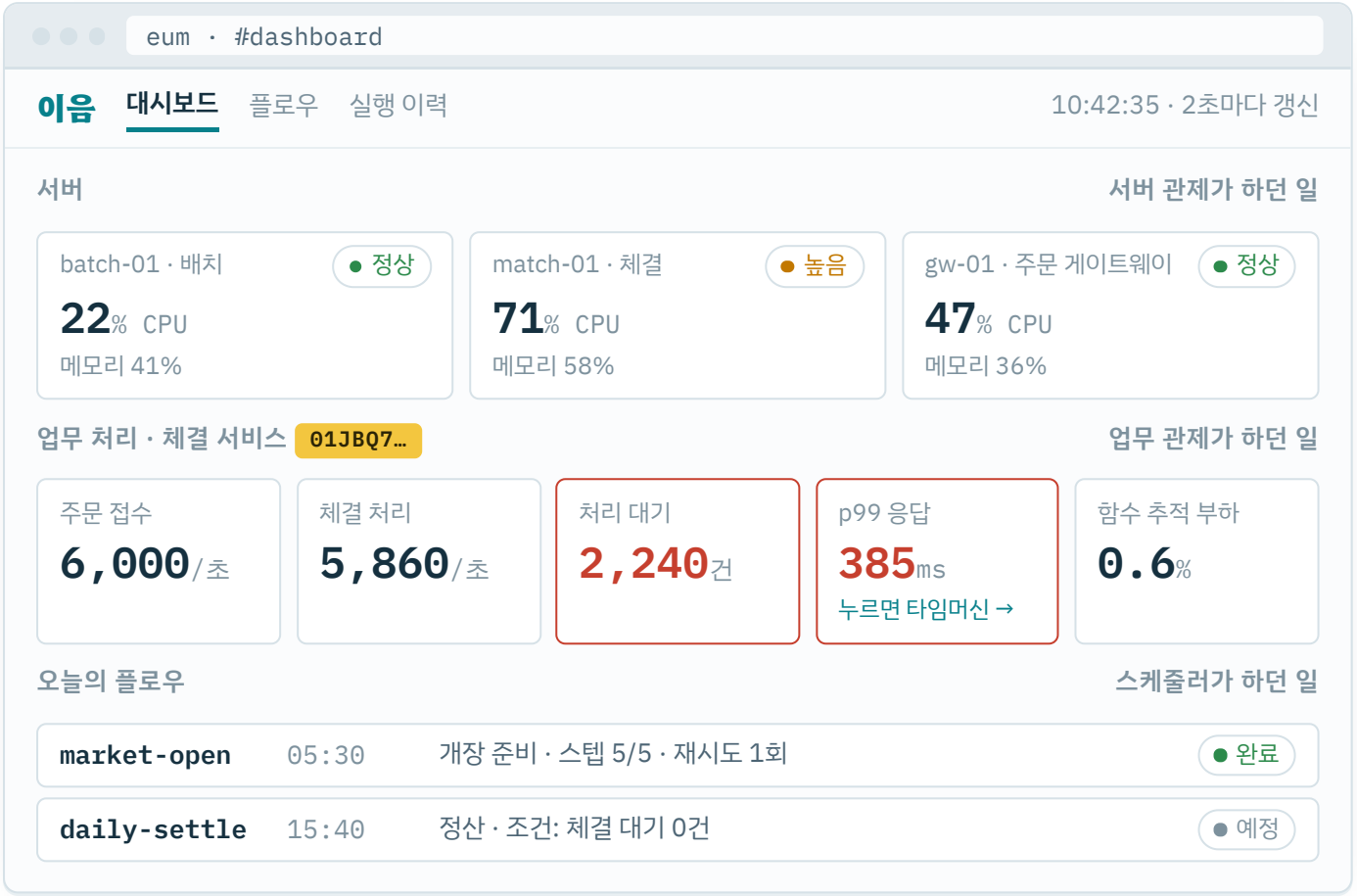
화면 ⑤ 데이터 꺼내기 · 12쪽

Q3 뭐가 좋아지는 건가요?

① 복잡도와 관리 비용

서버·업무·배치를 한 화면에.

지금은 도구를 여러 개 열어야 보이는 내용입니다.



설계 시안 · 수치는 합성 부하 예시

무엇을 보나

서버 자원, 주문 접수·처리·대기·응답시간, 오늘의 배치.

어떻게 쓰나

업무 지표에 실행 번호가 붙어 있습니다. 응답시간 타일을 누르면 해당 실행 기록으로 바로 갑니다.

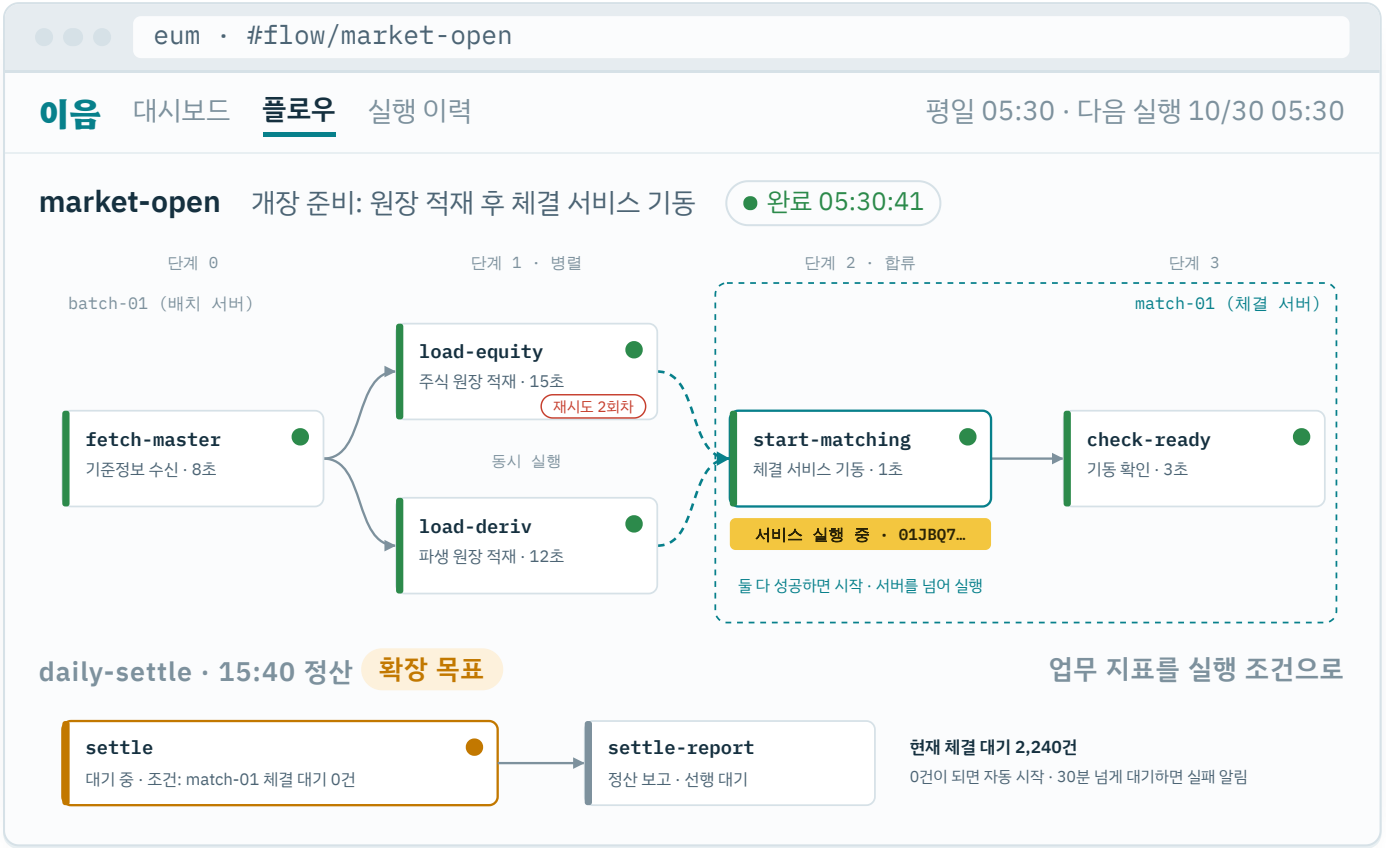
좋아지는 점

여러 화면을 오가던 점검이 화면 하나로.

Q3 뭐가 좋아지는 건가요? ② 배치를 유연하게

배치 흐름은 그래프로, 실행은 업무에 맞춰.

병렬·합류·서버 넘기·재시도를 한눈에 봅니다.



설계 시안 · 노드를 누르면 해당 실행 기록(타임머신)으로 이동

무엇을 보나

선후행, 병렬과 합류, 서버 간 실행, 재시도. 서비스를 띄운 스텝이 끝나도 서비스는 같은 번호로 계속 기록합니다.

업무를 아는 스케줄러

업무 지표가 같은 DB에 있어 "체결 대기 0건 뒤 정산" 같은 조건을 바로 겁니다. 지표가 끊기면 실행하지 않습니다.

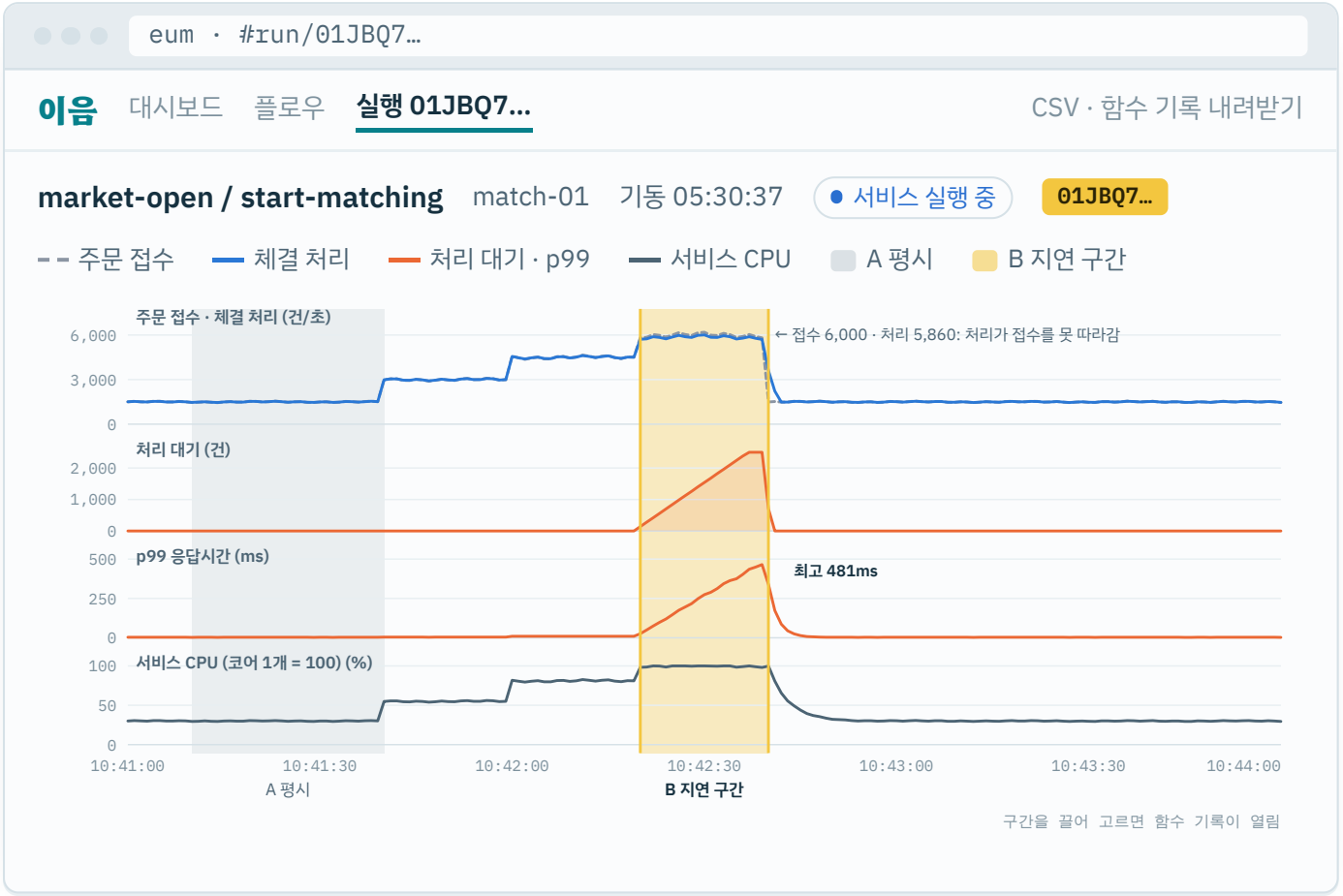
좋아지는 점

시각만 보던 배치가 업무 상황까지 봅니다.

Q3 뭐가 좋아지는 건가요? ③ 장애 원인을 한눈에

지나간 장애도 다시 펼쳐 봅니다.

주문이 초당 1,500건에서 6,000건으로 몰린 3분. 서비스의 실행 번호로 모아 한 시간축에 놓았습니다.



설계 시안 · 시연용 C 체결 서비스에 합성 주문으로 만든 부하 예시

무엇을 보나

서비스의 업무 지표, 서비스 CPU, 서버 자원을 한 시간축에. 도구별로 시각을 맞출 필요가 없습니다.

어떻게 쓰나

평시 구간과 지연 구간을 끌어서 고르면 다음 쪽 플레임 그래프가 열립니다.

좋아지는 점

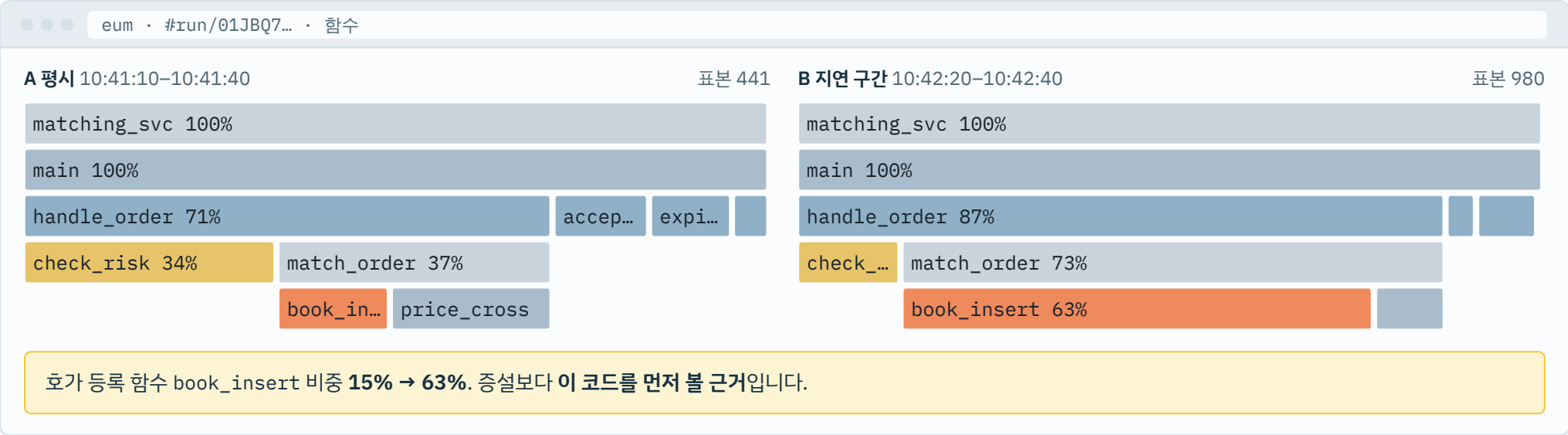
5단계 → 2단계. 타일 클릭, 구간 선택이면 함수까지.

단계 수는 절차 설계 기준. 소요 시간은 같은 시나리오로 실측해 비교합니다(13쪽).

Q3 뭐가 좋아지는 건가요? ③ 장애 원인을 한눈에

CPU를 쓴 함수까지 내려갑니다. 재현은 필요 없습니다.

위가 바깥 함수, 아래가 안쪽 함수. 막대 폭은 CPU 사용 비율입니다.



호가 등록 함수 book_insert 비중 **15% → 63%**. 증설보다 **이 코드를 먼저 볼 근거**입니다.

장애마다 보는 기록이 다릅니다

상황	함께 보는 기록	먼저 확인할 것
배치 실패	종료 코드, 로그 끝부분, 선행 작업 결과, 재시도 이력	어느 단계에서 왜 실패했나
서비스 중단	번호로 추적하던 프로세스 수, 직전 자원 변화	언제 사라졌고, 직전에 뭐가 있었나
응답 지연	처리량·대기·응답시간, 자원, 함수별 CPU	업무량·자원·코드 중 어디부터 볼까

범위

함수 기록은 CPU를 쓴 곳만 보여 줍니다. 잠금·디스크 대기는 이번 범위 밖입니다. CPU 비중만으로 원인을 단정하지 않습니다.

좋아지는 점

재현 환경을 만들기 전에 볼 함수가 좁혀집니다.

Q3 뭐가 좋아지는 건가요? ④ 데이터 마이닝

쌓인 기록을 SQL로 꺼내 마이닝합니다.

화면은 DB를 보는 방법 중 하나일 뿐입니다. 읽기 전용 SQL과 CSV 내보내기를 처음부터 기능으로 둡니다.

```
-- 최근 5영업일: 초당 몇 건부터 체결이 밀리나
$ eum sql "SELECT in_tps, seconds, avg_p99_ms FROM ..."
 in_tps | seconds | avg_p99_ms
--------+---------+-----------
   1500 | 109,560 |        3.4
   3000 |   6,420 |        4.1
   4500 |   1,140 |        9.8
   6000 |     180 |      254.1

-- 평시 vs 지연 구간, 함수별 CPU 비중(%)
-- 업무 지표와 함수 기록을 실행 번호로 조인
$ eum sql --csv "WITH w AS (...) SELECT ..." > slow.csv
 leaf          | normal_pct | slow_pct
---------------+------------+---------
 book_insert   |       15.2 |     63.4
 check_risk    |       34.0 |     14.0
 price_cross   |       21.8 |      9.6
 expire_orders |       11.1 |      8.2
```

설계 시안 · 두 질의는 같은 스키마의 합성 데이터로 실행 확인 · 수치는 예시

이렇게 씁니다

활용	엮는 데이터	얻는 것
용량 판단	업무 지표 + 자원	밀리기 시작하는 처리량 → 증설 시점
코드 개선	업무 지표 + 함수	지연 때 비중이 커지는 함수
배치 종료 예측	실행 이력 + 자원	정산 종료 시각 추세
이상 탐지	실행 이력	평소보다 오래 걸린 실행 표시
배포 전후 비교	실행 이력 + 함수	함수 비중 변화로 성능 저하 발견

데이터 주인은 우리

스키마 공개. 컬럼은 추가만 해서 기존 질의가 깨지지 않습니다. 보존 기간·수집 주기도 우리가 정합니다.

좋아지는 점

한 번 보고 끝나던 기록이 분석 재료가 됩니다.

Q3 뭐가 좋아지는 건가요? 확인 방법

효과는 숫자로 확인합니다.

확인 안 된 절감 수치는 쓰지 않습니다. 검증판이 상용 제품을 바로 대체한다고 가정하지도 않습니다.

확인 방법

좋아지는 것	확인 방법
① **복잡도·관리 비용**	기존 도구들의 점검 항목이 한 화면에 모두 나오는지 점검표로 확인
② **유연한 배치**	병렬·합류·서버 넘기·재시도가 정의대로 돌고 그래프에 보이는지
③ **장애 원인 파악**	같은 시나리오를 지금 절차와 이음으로 처리해 소요 시간 비교. 지난 구간의 함수 비중을 재현 없이 조회하는지
④ **데이터 마이닝**	운영 데이터 5종을 SQL로 조인하고 CSV로 내보내는지
수집 부담	추적 자체의 CPU, 서비스 처리량·응답시간 변화 측정

연간 절감 시간 = 월 조사 건수 [기입] × 건당 조사 시간 차이 × 12
연간 순절감액 = 실제 대체한 라이선스·유지보수 [기입] − 추가 운영·유지보수 비용 [기입]

초기 개발·전환 비용은 따로 넣어 총비용과 회수 기간을 봅니다. 건수·시간·계약 금액은 제출 전 사내 확인이 필요합니다.

적용 순서

1단계
가상 환경 검증
연결·과거 조회·함수 비교 확인

2단계
비운영 서버 시험
권한·빌드 조건·부하·장시간 안정성

3단계
제한적 병행 운영
일부 작업에 기존 도구와 함께 사용, 대체 범위 판단

운영 전 필수

로그인·권한, 암호화 통신(TLS), 복구·이중화, 수집 누락·보존 정책. 설계상 자리는 있고, 투입 전에 구현·검증합니다.

비용은 대체한 만큼만

병행 운영에서 실제로 대체한 기능만 절감액으로 셉니다. 유지보수 인건비, 저장 공간, 보안 점검 비용도 함께 넣습니다.

원칙

작게 검증하고, 확인된 만큼 넓힙니다.

Q4 어떻게 가능한 건가요?

실행 번호 하나가 모든 기록을 잇는 키입니다.

새로 발명하는 기술은 없습니다. 검증된 부품을 쓰고, 부품 사이를 잇는 코드만 만듭니다.

번호가 이어지는 과정

1 번호 발급
스케줄러가 실행마다 고유 번호를 발급. 재시도는 새 번호.

2 명령과 함께 전달
에이전트가 명령과 번호를 받아 스크립트 실행.

3 프로세스에 상속
환경변수로 넘긴 번호가 스크립트가 띄운 C 서비스까지 따라감. 스크립트가 끝나도 남음.

4 번호로 찾아 기록
에이전트가 번호를 가진 프로세스의 자원·함수를 기록. 서비스는 업무 지표에 번호를 붙여 보냄.

실행 결과	서비스 자원	업무 지표	함수 기록
시작·종료·재시도	CPU·메모리, 1초	접수·처리·대기·p99	10초 단위
01JBQ7…	01JBQ7…	01JBQ7…	01JBQ7…

어려운 일은 검증된 부품이

할 일	쓰는 부품	우리가 만드는 부분
함수 기록	리눅스 커널 내장 perf	대상 프로세스 선택, 결과 압축 저장
저장·조회	SQLite(파일 하나짜리 DB)	공개 스키마, 분석용 뷰, 조회 API
실행 시각	검증된 cron 해석 라이브러리	선후행·재시도 판단, 실행 루프
차트	uPlot(경량 차트)	플로우 그래프, 플레임 그래프
배포	Go 단일 실행 파일	매니저·에이전트 역할 분리

구성

batch-01 배치 서버 에이전트
match-01 체결 서버 에이전트
gw-01 게이트웨이 에이전트

→

매니저 스케줄러·수집·저장·화면
실행 상태 DB + 지표·함수 DB
연결은 에이전트 → 매니저 한 방향

Q4 어떻게 가능한 건가요? 하루, 2명

기반은 하루에 입증하고, 기능은 계속 얹습니다.

2명이 AI 세션 4개로 단순한 아키텍처의 기반을 구현·입증합니다. 코드는 대회 당일 처음부터 쓰고, 사전 준비는 설계 문서·작업카드·AI 지시문뿐입니다.

당일 일정



범위

15:00까지 필수

시각 실행, 선후행·병렬·합류·재시도, 원격 실행, 서비스 추적, 자원·업무 지표 수집, 함수 기록, 타임머신·플레임 그래프, SQL·CSV, 에이전트 토큰

시간이 남으면

업무 지표 조건 실행(정산 대기), 함수 비교 표, 임계치 알림, 실행 강제 종료

이번에 안 함

로그인·권한, TLS, 이중화, 휴일 캘린더, 자바 함수 추적, 잠금·디스크 대기 분석

미리 확인한 것

함수 이름 복원	운영과 같은 빌드 조건(-O2, 프레임 포인터 없음)의 C 시험 프로그램에서 호출 경로 정확히 복원
추적 부담	코어 1개의 **약 0.6%**, 대상 소요 시간 변화는 오차 범위. 시험 프로그램 기준이라 본선에서 다시 재서 화면에 띄움
서비스 추적	스크립트 밖으로 나간 프로세스를 환경변수 표식으로 찾음(일반 사용자 권한)
분석 질의	12쪽 SQL 2개를 합성 데이터로 실행
운영 서버	perf 사용 가능

기능은 끼워 넣는 구조

수집·추적·함수 기록·저장·알림을 인터페이스 6개로 나눕니다. 자바 함수 기록, 시계열 DB, 메신저 알림은 구현만 추가해 붙입니다.

늦어지면 줄이는 순서

대시보드 장식 → 내보내기 → 구간 끌어 고르기(고정 구간으로) → 플레임 그래프(상위 함수 표로). 번호 연결과 함수 수집은 유지합니다.

본선은 외부망·상용 AI로 개발하고 사내 데이터·소스는 쓰지 않습니다. 시연은 컨테이너 3대와 C 체결 서비스로 하며, 서버 자원만 모의 값입니다.